

You are a senior software architect helping me build a hackathon project called **AI Investment Arena**.

I want you to generate the **initial repository architecture and development skeleton**, NOT the complete application yet.

The project is a 0-budget / free-tier hackathon demo, so architecture must prioritize:

- Free/open-source technologies
- Minimal API/LLM usage
- Avoiding token exhaustion
- Graceful degradation
- Local Docker development
- Easy deployment later
- Clear separation of concerns
- Ability to develop each service independently

Do NOT over-engineer the system.

---

# 1. PRODUCT

The platform is an AI-powered startup investment marketplace.

It combines:

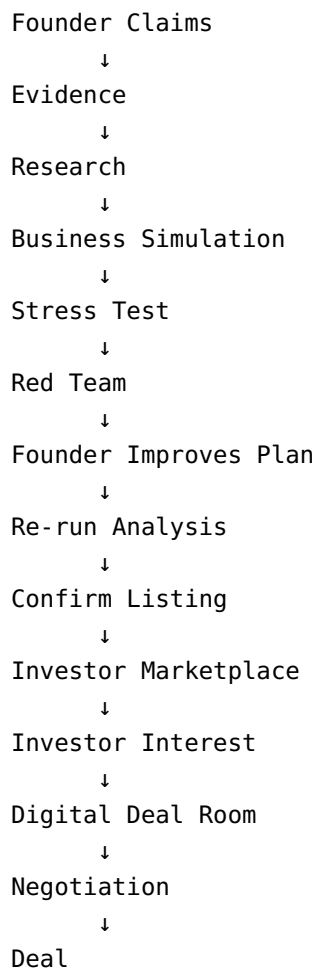
- Startup registration
- Startup profiles
- Investor registration
- Investor profiles
- Startup document uploads
- AI document/evidence analysis
- AI background research
- Market research
- Competitor analysis
- Business-plan analysis
- Financial/business simulation
- Bull/base/bear scenarios
- Red-team analysis
- AI investment intelligence report
- AI investor/startup matching
- Deal marketplace
- Investor interest
- Multiple simultaneous deal rooms
- Negotiation
- AI negotiation copilot

- Deal/cap-table calculations

The core differentiator is:

**AI does not simply judge whether a startup sounds good. It converts the founder's claims and assumptions into a business model, researches supporting evidence, stress-tests the assumptions, simulates possible futures, and gives investors evidence-backed investment intelligence.**

The core loop is:



## 2. TECHNOLOGY REQUIREMENTS

### Frontend

Use:

- React
- JavaScript
- Vite
- Tailwind CSS
- React Router
- Axios or fetch
- No TypeScript
- No unnecessary UI framework unless absolutely necessary

Create:

```
frontend/
```

The frontend must communicate with backend APIs rather than directly implementing business logic.

---

## 3. Backend

Use microservices.

Use Python + FastAPI for backend services.

Create:

```
backend/
```

Do NOT create excessive microservices.

For the initial architecture, use these services:

```
backend/  
├─ api-gateway/  
├─ user-service/  
├─ startup-service/  
└─ investor-service/
```

```
|— deal-service/  
|— ai-service/
```

Explain why each service exists.

---

## 4. API Gateway

The API gateway should be the main backend entry point.

Responsibilities:

- Route requests
- Authentication verification
- Basic request validation
- Forward requests to internal services
- Hide internal service URLs from frontend
- Basic rate limiting if practical

Frontend should communicate primarily with:

```
/api/*
```

rather than directly calling every microservice.

Do not turn the gateway into a complicated enterprise gateway.

---

## 5. Supabase

Use Supabase for:

- Authentication
- PostgreSQL
- File/object storage
- User identity
- Basic relational application data

Authentication must support:

```
Startup  
Investor
```

Use Supabase Auth.

The backend should verify Supabase JWTs.

Do not build a custom authentication system.

---

## 6. Suggested Supabase data

Design the initial schema around:

```
profiles
startups
investors
startup_documents
startup_claims
startup_analysis_runs
startup_metrics
investment_preferences
deals
deal_interests
deal_rooms
deal_messages
offers
negotiations
```

Do not blindly create every possible table.

Provide a database schema proposal and explain relationships.

Important:

A user should have one identity in Supabase Auth, with a profile indicating whether they are a startup user or investor.

---

## 7. Neo4j

Use Neo4j as the relationship database.

Do NOT duplicate the entire Supabase database into Neo4j.

Neo4j should be used only for relationships such as:

### Startup

- |— founded\_by → Founder
- |— operates\_in → Industry
- |— competes\_with → Competitor
- |— targets → Market
- |— claims → Claim
- |— supported\_by → Evidence
- |— interested\_by → Investor
- |— similar\_to → Startup

### Investor

- |— interested\_in → Industry
- |— invested\_in → Startup
- |— prefers → Stage
- |— connected\_to → Founder

Potential graph:

```
(:Startup)-[:OPERATES_IN]->(:Industry)
(:Startup)-[:COMPETES_WITH]->(:Startup)
(:Startup)-[:TARGETS]->(:Market)
(:Startup)-[:MAKES_CLAIM]->(:Claim)
(:Claim)-[:SUPPORTED_BY]->(:Evidence)
(:Investor)-[:INTERESTED_IN]->(:Industry)
(:Investor)-[:INTERESTED_IN]->(:Startup)
```

Neo4j should help with:

- Relationship exploration
- Competitor relationships
- Evidence/claim relationships
- Investor/startup matching
- Similar startups
- Graph visualization later

Supabase remains the primary transactional database.

---

## 8. Google ADK

Use **Google ADK wherever it provides real value.**

Do not force ADK into ordinary CRUD operations.

The AI system should live primarily inside:

```
backend/ai-service/
```

Use Google ADK for the agent orchestration layer.

Suggested architecture:

```
AI Orchestrator
|
├─ Document Intelligence Agent
├─ Market Research Agent
├─ Competition Agent
├─ Financial Analysis Agent
├─ Business Simulation Agent
├─ Red Team Agent
├─ Investment Analyst Agent
└─ Negotiation Copilot Agent
```

However, do NOT execute every agent for every request.

Create workflows so that only necessary agents run.

---

## 9. Agent responsibilities

### Document Intelligence Agent

Input:

- Uploaded documents
- Founder claims

Responsibilities:

- Extract relevant information
- Compare claims against documents
- Detect mismatches
- Identify missing evidence
- Categorize evidence

Output should be structured JSON such as:

```
{
  "claim": "Annual revenue is ₹40 lakh",
  "status": "partially_verified",
  "evidence": "financial_statement.pdf",
  "confidence": 0.82,
  "reason": "Document supports ₹31.7 lakh identifiable revenue"
}
```

IMPORTANT:

Never claim that AI has legally authenticated a government document.

Use statuses such as:

```
SUPPORTED
PARTIALLY_SUPPORTED
UNVERIFIED
CONTRADICTED
INSUFFICIENT_EVIDENCE
```

---

## 10. Market Research Agent

Responsibilities:

- Market size
- Market growth
- Industry trends
- Customer pain points
- Competitors
- Competitive threats
- Market opportunities

If external web research is used, design the service so the research provider can be replaced.

Do NOT make the entire application dependent on an expensive search API.

For the 0-budget demo, allow:

```
MOCK_RESEARCH=true
```

and optionally support free/public sources.

---

## 11. Financial Analysis Agent

This agent should interpret founder assumptions.

It should NOT perform all calculations itself through LLM reasoning.

For example, the LLM can identify:

```
monthly_growth = 0.18
CAC = 600
churn = 0.04
marketing_budget = 500000
```

Then deterministic Python code should calculate:

- Revenue
- Customers
- CAC
- Churn
- Burn
- Runway
- Gross margin
- Funding requirements
- Dilution
- Valuation scenarios

This is extremely important.

Use normal Python functions for mathematical calculations.

Do NOT ask the LLM to calculate complex financial projections.

---

## 12. Business Simulation Engine

Create a deterministic simulation module.

For example:

```
simulation/
├─ assumptions.py
├─ revenue_model.py
```

```
|— customer_model.py
|— cost_model.py
|— runway_model.py
|— valuation_model.py
|— scenarios.py
└— simulator.py
```

Support:

```
Bull Case
Base Case
Bear Case
```

Eventually allow probabilistic / Monte-Carlo-style simulations.

But keep the first implementation simple.

The simulator should return structured JSON.

Example:

```
{
  "bull": {
    "revenue_18_months": 12400000,
    "customers": 320000,
    "runway_months": 18
  },
  "base": {
    "revenue_18_months": 7100000,
    "customers": 140000,
    "runway_months": 12
  },
  "bear": {
    "revenue_18_months": 2900000,
    "customers": 42000,
    "runway_months": 7
  }
}
```

---

## 13. Red Team Agent

This is one of the most important AI components.

The Red Team Agent should attempt to challenge the startup's thesis.

Examples:

```
Founder says:
"We will achieve 30% monthly growth."

Red Team:
"What acquisition volume is required?"

Founder says:
"TAM is ₹50,000 Cr."

Red Team:
"What evidence supports the serviceable market?"

Founder says:
"We will acquire customers for ₹200."

Red Team:
"Does the current acquisition model support this CAC?"
```

Output:

```
{
  "severity": "high",
  "issue": "Customer acquisition assumption",
  "explanation": "...",
  "impact": "Runway decreases by approximately 6 months",
  "recommendation": "..."
}
```

The Red Team should challenge assumptions rather than merely produce generic criticism.

---

## 14. Founder iteration workflow

This is critical.

The startup should be able to:

```
Create thesis
↓
```

```
Run AI analysis
  ↓
View results
  ↓
Edit assumptions
  ↓
Run analysis again
  ↓
Compare versions
  ↓
Confirm listing
```

Store analysis versions.

For example:

```
analysis_version_1
analysis_version_2
analysis_version_3
```

Once the founder confirms listing:

```
listing_status = CONFIRMED
```

Freeze that analysis version for the marketplace.

---

## 15. AI Investment Report

Generate a structured report containing:

```
Company Overview
Business Model
Market Analysis
Competition
Document Evidence
Claim Verification
Financial Analysis
Bull/Base/Bear Simulation
Key Risks
Key Opportunities
Red Team Findings
```

Investment Readiness  
Founder Projection vs AI Projection

Avoid generating huge prose reports.

Store structured data first.

Generate summaries from the structured data.

This is important for reducing LLM token consumption.

---

## 16. Investor system

Investor onboarding:

```
Register
  ↓
Create profile
  ↓
Upload CV/profile document
  ↓
AI extracts experience
  ↓
Background research
  ↓
Investor profile generated
```

Again, do not claim legal identity verification.

Call this:

AI Background Assessment

Store:

```
experience
industries
roles
investment_preferences
check_size
risk_appetite
```

```
public_evidence
verification_status
```

---

## 17. Deal marketplace

Once a startup confirms its listing, it appears in:

```
/deals
```

Investors can filter by:

```
Industry
Stage
Amount
Equity
Royalty
AI score
Risk
Revenue
Growth
Listing date
```

Create APIs for:

```
list deals
get deal
filter deals
sort deals
recommended deals
```

---

## 18. Investor matching

Create a deterministic matching engine first.

Do NOT use an LLM for every match.

Calculate a score based on:

Industry  
Stage  
Check size  
Geography  
Risk appetite  
Business model  
Investor preferences

Example:

|                   |    |
|-------------------|----|
| Industry match    | 25 |
| Stage match       | 20 |
| Check size        | 20 |
| Geography         | 10 |
| Risk alignment    | 10 |
| Business model    | 10 |
| Portfolio synergy | 5  |

Then optionally use an LLM to explain the score.

This reduces token usage.

---

## 19. Digital Deal Room

A startup can have multiple investors.

An investor can have multiple active deal rooms.

Therefore:

Startup

- └ Deal Room A → Investor A
- └ Deal Room B → Investor B
- └ Deal Room C → Investor C

Investor A

- └ Deal Room → Startup X
- └ Deal Room → Startup Y
- └ Deal Room → Startup Z

Deal room features:

- Messages
- Offers
- Counteroffers
- Equity
- Investment amount
- Royalty
- Notes
- Status
- Timeline

Example statuses:

INTERESTED  
NEGOTIATING  
COUNTERED  
ACCEPTED  
REJECTED  
CLOSED

---

## 20. Negotiation Copilot

The AI should assist both sides.

It should NOT autonomously make deals.

For startup:

"Is this offer reasonable?"

For investor:

"How does this offer compare with the AI valuation?"

AI can calculate:

Investment amount  
Equity  
Implied valuation

Dilution  
Runway extension

The LLM should explain the numbers, while deterministic Python calculates them.

Example:

Investor:  
₹1Cr for 10%

Implied post-money valuation:  
₹10Cr

Pre-money:  
₹9Cr

Do not let an LLM perform this calculation.

---

## 21. AI TOKEN / COST CONTROL — VERY IMPORTANT

The entire application must be designed for a **0-budget hackathon demo**.

This is one of the most important requirements.

Design a centralized AI usage system.

Create:

```
ai-service/  
├─ agents/  
├─ workflows/  
├─ tools/  
├─ prompts/  
├─ schemas/  
├─ cache/  
├─ usage/  
└─ providers/
```

Create an AI provider abstraction:

AIProvider

- ☐ GeminiProvider
- ☐ GroqProvider
- ☐ MockProvider

The application should be able to switch providers using environment variables.

Example:

AI\_PROVIDER=gemini

or:

AI\_PROVIDER=groq

or:

AI\_PROVIDER=mock

## 22. AI reliability architecture

The system MUST NOT crash when an AI provider fails.

Implement:

```
LLM Request
  ↓
Cache lookup
  ↓
Existing result?
  ├── YES → return cached result
  └── NO
      ↓
    Call LLM
      ↓
    Validate structured output
      ↓
    Store result
```

If the LLM fails:

```
LLM failure
  ↓
Retry with exponential backoff
  ↓
Second failure
  ↓
Fallback provider
  ↓
Second provider fails
  ↓
Use cached result / deterministic fallback
```

Do not retry indefinitely.

Use a strict retry limit.

---

## 23. Token minimization

Design agents so they:

- Receive only necessary context
- Never receive entire documents repeatedly
- Use structured JSON instead of huge text
- Summarize documents once
- Cache summaries
- Cache research results
- Cache analysis results
- Reuse previous analysis
- Send only changed assumptions during re-analysis where possible
- Avoid unnecessary multi-agent conversations
- Avoid agents talking to each other endlessly

DO NOT implement:

```
Agent A → Agent B → Agent C → Agent A → Agent B → ...
```

without strict limits.

Each workflow should have a maximum number of agent calls.

---

## 24. Important AI architecture rule

Separate:

### **Deterministic computation**

Use Python for:

```
financial calculations  
simulation  
valuation  
dilution  
matching score  
filtering  
sorting  
risk calculations
```

### **AI reasoning**

Use LLM/ADK for:

```
document interpretation  
claim extraction  
market reasoning  
competitive analysis  
risk explanation  
red teaming  
natural-language explanations  
negotiation assistance
```

This should be clearly reflected in the repository architecture.

---

## 25. Async AI jobs

Do not make the frontend wait for a 2-minute AI workflow.

Use background jobs.

For example:

```
POST /startup/{id}/analysis
↓
Create analysis_job
↓
Return job_id
↓
Background worker
↓
ADK workflow
↓
Store results
↓
status = COMPLETED
```

Frontend polls:

```
GET /analysis/{job_id}
```

Possible statuses:

```
QUEUED
RUNNING
COMPLETED
FAILED
```

For the first version, use a lightweight background task / worker rather than immediately adding Kafka or RabbitMQ.

The architecture should allow a real queue to be added later.

---

## 26. Docker

Everything must be Dockerized.

Root:

```
docker-compose.yml
```

Services should include:

```
frontend
api-gateway
user-service
startup-service
investor-service
deal-service
ai-service
neo4j
```

Supabase should preferably be accessed through its hosted free tier rather than running the entire Supabase stack locally.

Do not containerize Supabase unless there is a strong reason.

Use environment variables.

---

## 27. Development commands

The final architecture must support:

```
docker compose up
```

```
docker compose up --build
```

```
docker compose down
```

```
docker compose logs -f
```

And preferably:

```
docker compose up frontend api-gateway
```

for partial development.

Use Docker volumes where appropriate.

Make hot reload work for frontend/backend development if practical.

---

## 28. Environment variables

Create:

```
.env.example
```

Include placeholders for:

```
SUPABASE_URL=  
SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
  
NEO4J_URI=  
NEO4J_USERNAME=  
NEO4J_PASSWORD=  
  
AI_PROVIDER=gemini  
  
GEMINI_API_KEY=  
GROQ_API_KEY=  
  
SEARCH_PROVIDER=  
SEARCH_API_KEY=  
  
FRONTEND_URL=
```

Never hard-code secrets.

Never commit `.env`.

Create appropriate `.gitignore` files.

---

## 29. Mock mode

Because this is a hackathon and APIs can fail, implement:

```
DEMO_MODE=true
```

When enabled:

- AI responses can use deterministic mock responses
- Market research can use seeded sample data
- Document analysis can use sample structured outputs
- Investor recommendations can use seeded data

This ensures that the demo **cannot completely collapse because of an external API failure**.

The real AI mode should still work when keys are provided.

---

## 30. Demo dataset

Create a seed system.

For example:

```
scripts/  
seed_demo_data.py
```

Create:

```
3 startups  
5 investors  
competitors  
industries  
sample claims  
sample evidence  
sample deals
```

The application should be demoable immediately.

---

## 31. Observability

Create simple logging.

Every AI workflow should log:

```
workflow  
startup_id
```

```
agent
start_time
end_time
status
provider
cached
error
```

Do not log sensitive document contents.

Create an AI usage tracker:

```
ai_requests
tokens_if_available
estimated_cost
cache_hits
failures
```

This will help prevent accidental API quota exhaustion.

---

## 32. Security

Implement basic security:

- Supabase JWT verification
- Backend authorization
- Startup users can only modify their own startup
- Investors can only modify their own profile
- Deal room participants can access their deal room
- Do not expose service-role Supabase keys to frontend
- Validate uploaded files
- Limit upload size
- Do not execute uploaded files
- Sanitize extracted text
- Never expose API keys

This is a hackathon demo, so keep security practical rather than implementing enterprise IAM.

---

## 33. Repository structure

Generate a clean initial structure approximately like:

```
ai-investment-arena/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── layouts/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── context/
│   │   ├── utils/
│   │   └── App.jsx
│   ├── Dockerfile
│   ├── package.json
│   └── vite.config.js
├── backend/
│   ├── api-gateway/
│   │   ├── app/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── user-service/
│   │   ├── app/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── startup-service/
│   │   ├── app/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── investor-service/
│   │   ├── app/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── deal-service/
│   │   ├── app/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   └── ai-service/
│       ├── app/
│       │   ├── agents/
│       │   └── workflows/
```

```
├── tools/
│   ├── prompts/
│   ├── schemas/
│   ├── simulation/
│   ├── providers/
│   ├── cache/
│   └── main.py
├── Dockerfile
└── requirements.txt

├── database/
│   ├── supabase/
│   │   ├── migrations/
│   │   └── seed/
│   └── neo4j/
│       └── seed/
├── scripts/
│   └── seed_demo_data.py
├── docs/
│   ├── architecture.md
│   ├── api.md
│   └── ai-workflows.md
├── .env.example
├── .gitignore
├── docker-compose.yml
├── README.md
└── LICENSE
```

You may improve this structure if you have a strong architectural reason.

### 34. What I want you to generate NOW

Do NOT build all features yet.

Generate only the **initial repository skeleton**.

I want:

1. Complete directory tree
2. Docker Compose configuration
3. Dockerfiles
4. FastAPI service skeletons

5. React/Vite/Tailwind skeleton
6. Basic API Gateway
7. Basic Supabase integration structure
8. Basic Neo4j connection structure
9. Google ADK integration skeleton
10. AI provider abstraction
11. Mock AI provider
12. AI caching structure
13. Simulation module skeleton
14. Basic health endpoints
15. Environment configuration
16. README with setup instructions
17. `.env.example`
18. Basic seed structure
19. Basic database schema proposal
20. Architecture documentation

Do NOT implement:

- Complete UI
- Complete authentication pages
- Complete AI agents
- Complex financial simulation
- Real web scraping
- Complete negotiation system
- Payment system
- Production deployment
- Advanced RBAC
- Kubernetes
- Kafka
- Redis unless genuinely necessary
- Celery unless genuinely necessary

We want a clean starting point.

---

## 35. Architecture priorities

Rank these priorities in this order:

1. Demo reliability
2. 0-budget/free-tier compatibility
3. AI failure tolerance
4. Low token consumption
5. Simple development workflow
6. Clean architecture
7. Extensibility

#### 8. Production scalability

Do NOT optimize for enterprise-scale traffic.

We need to win a hackathon demo.

---

## 36. Final requirement

Before generating files, briefly explain:

1. Why these microservices exist
2. Why Supabase + Neo4j are both being used
3. Why Google ADK is isolated in the AI service
4. How token usage is controlled
5. How the system survives LLM/API failure
6. Why deterministic Python simulation is separate from LLM reasoning
7. How `docker compose up` starts the complete stack

Then generate the repository.

Make the generated code **minimal, runnable, and understandable**.

Do not create fake implementations that pretend to work.

If a component is only a placeholder, clearly mark it as a placeholder.

The result must be something I can clone/open in VS Code and immediately start developing.